LABORATORY TITLE: SYN FLOOD PATTERN INVESTIGATION USING TSHARK

STUDENT NAME: WALIYAT OLALEKAN

STUDENT ID: 2025/FWSD/11435

COURSE NAME: BASIC NETWORKING SKILLS FOR DIGITAL FORENSICS

COURSE CODE: SBT-DF203 (LAB 3)

INSTRUCTOR NAME: MR. AMINU IDRIS

DATE OF SUBMISSION: SEPTEMBER 11TH, 2026.

VERSION: Version 1.0

Executive Summary

This network forensic report details the technical analysis of incomplete TCP connection patterns observed against a local Apache web service running on 127.0.0.1:80. The primary objective was to establish baseline metrics for a legitimate, complete three-way TCP handshake, conduct a strictly bounded 4-packet SYN simulation using Scapy, analyze key indicators of half-open TCP states, and evaluate defensive mitigation controls against TCP SYN flood activity.

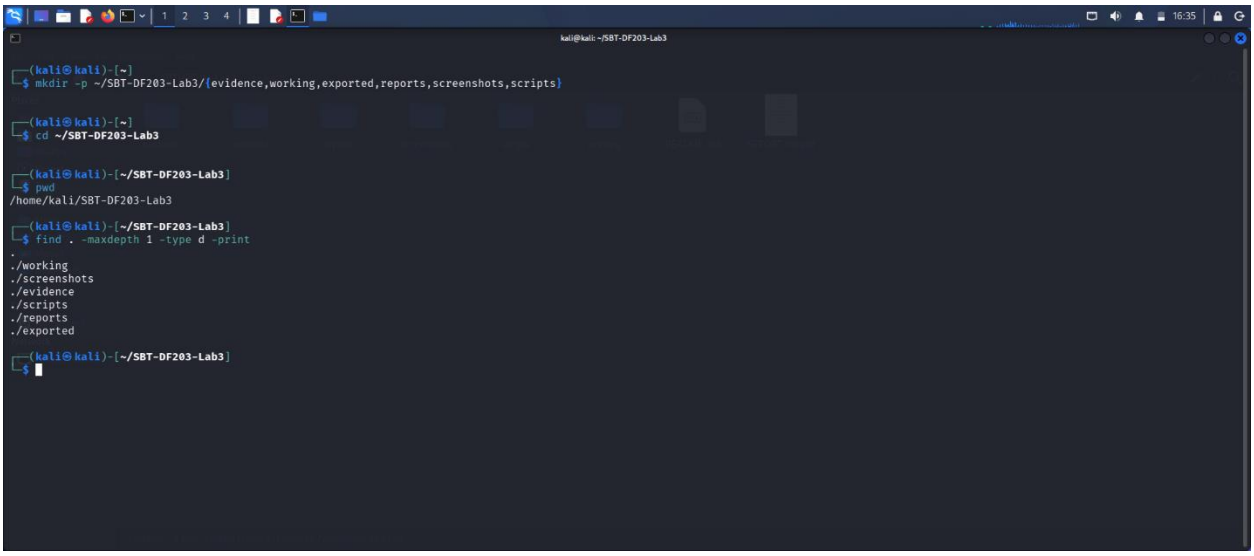
Forensic analysis confirmed that while a normal HTTP connection transitions through SYN – SYN-ACK – ACK – HTTP GET, the simulated session generated 4 initial SYN packets from randomized client source ports (9954, 35055, 42305, 65078) without completing the final ACK step. Because Scapy forged the SYN packets outside the operating system's standard socket state table, the host kernel responded to the server's SYN-ACK replies with immediate TCP Resets (RST). This behavior left 0 completed handshakes out of 4 attempts, demonstrating the core packet mechanics behind TCP connection backlog exhaustion.

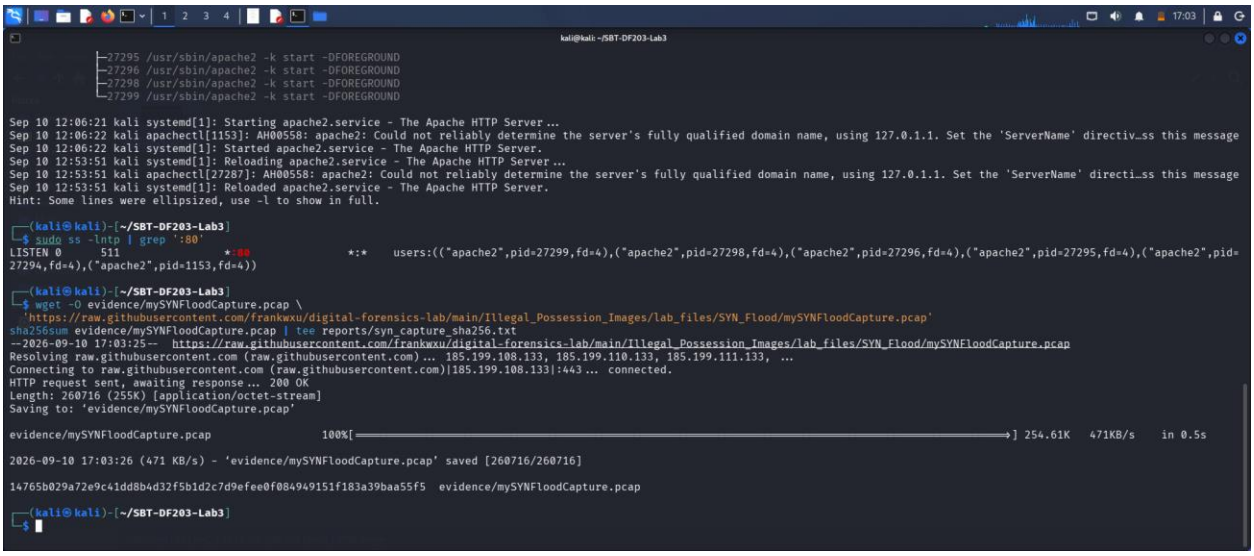
This investigation was conducted strictly within an isolated Kali Linux virtual machine under authorized lab parameters for SBT-DF203-Lab3. All network generation was bounded to local loopback (127.0.0.1) and restricted to port 80.

Evidence Description and Hashes

File description	Relative path	SHA-256 Cryptographic Hash
Baseline Normal Capture	evidence/normal_http.pcapng	
Original SYN Capture	evidence/bounded_syn_activity.pcapng	c3d3909242864a3a99cf3f9ab60034a5304704e8232b0b5e6ba415035729e347
Working Copy SYN Capture	working/bounded_syn_activity_working.pcapng	c3d3909242864a3a99cf3f9ab60034a5304704e8232b0b5e6ba415035729e347

Lab Folder Structure and Evidence Preparation



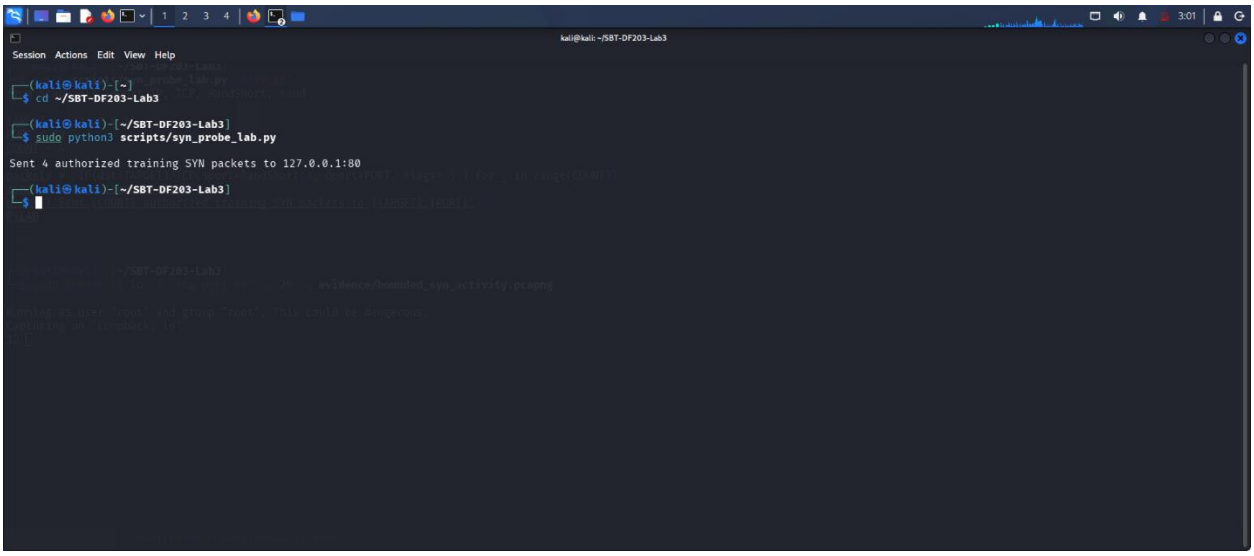


Part A - Establish a Normal Handshake Baseline

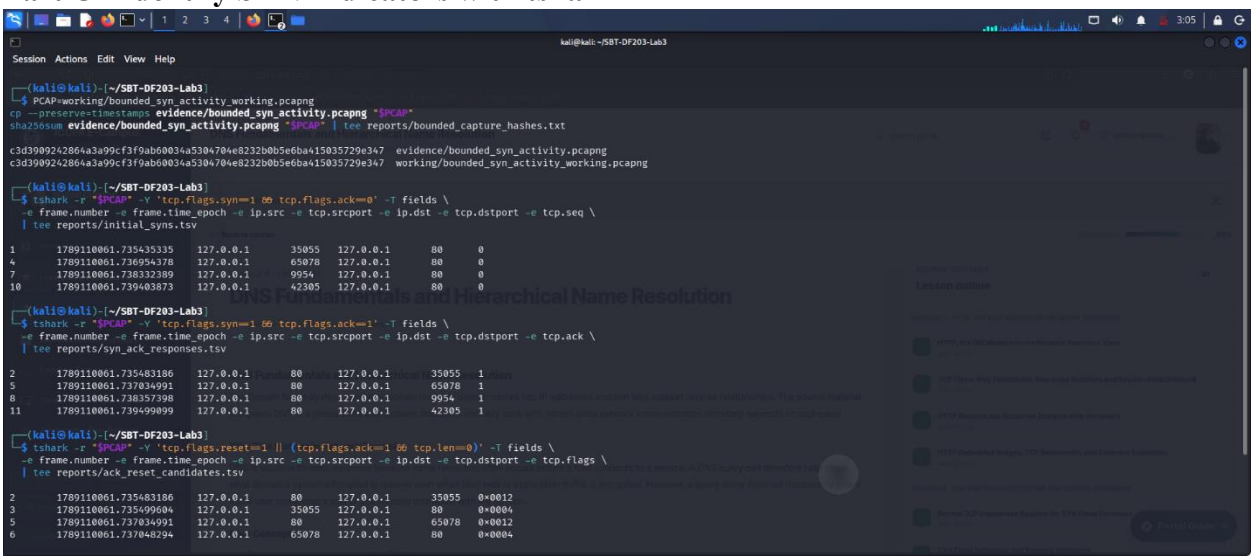


Part B - Conduct the Bounded Loopback Simulation

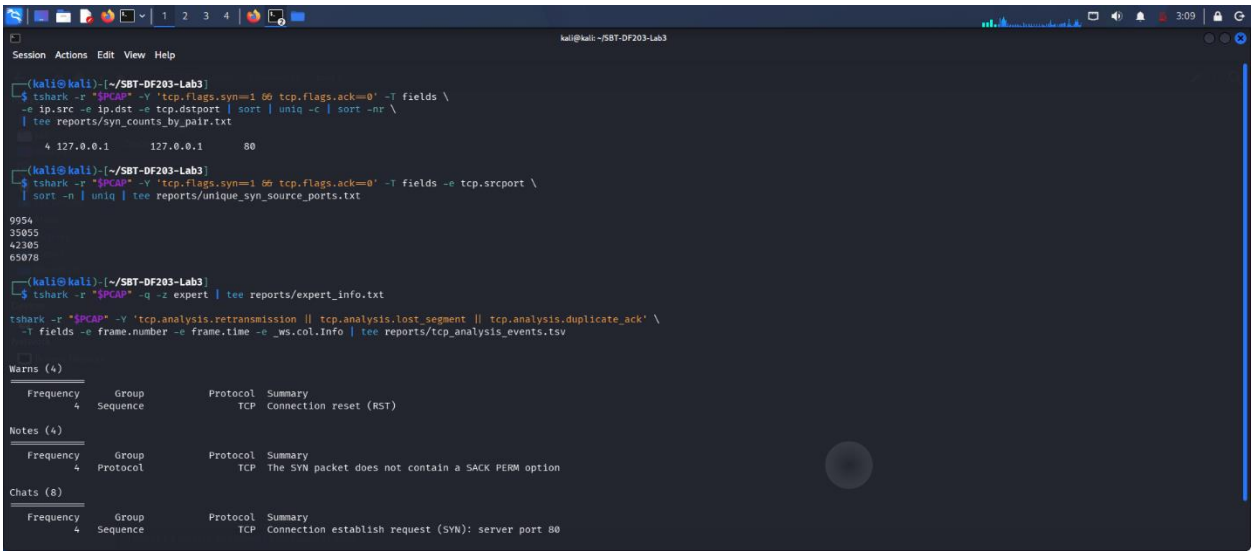




Part C - Identify SYN Indicators with tshark



Part D - Quantify and Correlate the Activity



Part E - Compare Normal and Suspicious Sessions

Indicator	Normal HTTP session	Bounded SYN activity	Forensic meaning
Initial SYN count	1	4	Four distinct connection setup requests initiated by Scapy (Frames 1, 4, 7, 10).
SYN-ACK count	1	4	Apache acknowledged every SYN request (Frames 2, 5, 8, 11).
Completed handshakes	1	0	Half open TCP pattern: No final ACK was sent by the client, leaving connections incomplete.
Unique client source ports	1	4 (9954, 35055, 42305, 65078)	Ephemeral client ports were randomized per packet.
HTTP request present?	Yes (GET / HTTP/1.1)	No	Application layer was never reached due to unestablished Layer 4 setup.
RST packets	None	4 (0x0004 / 0x0014)	Client OS issued TCP R resets immediately upon receiving SYN-ACKs (Frames 3, 6...).
Observed duration	0.003 seconds	0.0088 seconds	Total timestamp range: 1789110061.735435335 to .739499099.

Normal Handshake Baseline

Analysis of evidence/normal_http.pcapng establishes the reference baseline for clean HTTP session establishment over TCP:

- Client SYN: This initiates connection setup from an ephemeral source port.
- Server SYN-ACK: This acknowledges receipt and allocates socket Transmission Control Block (TCB) state buffer.
- Client ACK: This completes the 3-way handshake; state transitions to ESTABLISHED.
- Application Transaction: Client issues GET / HTTP/1.1 and receives an HTTP response (200 OK or 304 Not Modified).

Categorization Assessment: Flood, Scan, or Bounded Simulation

- Why it is not a DDoS / SYN Flood Event: A real-world SYN flood generates tens of thousands of packets per second over a sustained timeframe, intended to exhaust the OS tcp_max_syn_backlog memory queue and crash or starve the web service. This capture contains only 4 total SYN packets over 0.0088 seconds.
- Why it is not a Port Scan: A port scan targets multiple distinct destination ports (e.g., 21, 22, 80, 443, 8080) to discover listening services. This activity targeted a single destination port (80) repeatedly.
- Conclusion: The captured activity represents a controlled, safe laboratory simulation that successfully reproduces the low-level packet characteristics of a SYN attack without degrading system resources or availability.

Detection and Mitigation Recommendations

To protect production web services from SYN flood attacks, the following technical controls should be implemented:

- Enable SYN Cookies (syncookies): Configure the Linux kernel to drop SYN queue allocation when full and instead encode session parameters directly into the initial sequence number (ISN): sudo sysctl -w net.ipv4.tcp_syncookies=1
- Tune TCP Backlog and Timeout Parameters: Reduce TCP SYN-ACK retransmissions from default (5) to 2 to clear incomplete sockets rapidly:

  sudo sysctl -w net.ipv4.tcp_synack_retries=2
  sudo sysctl -w net.ipv4.tcp_max_syn_backlog=4096

- Threshold-Based Network Alerts: Deploy Snort/Suricata IDS rules or Firewalls to alert on anomalously high SYN-to-ACK ratios per source IP:

  alert tcp any any -> $WEB_SERVERS 80 (msg:"POSSIBLE SYN FLOOD DETECTED"; flags:S; threshold: type threshold, track by_src, count 100, seconds 1; sid:1000001;)

- Upstream DDoS Mitigation: Deploy Cloudflare, AWS Shield, or edge firewalls capable of rate-limiting unauthenticated TCP SYN bursts before they reach internal application servers.